CCIP503 Computer Networks & Security

Week – 5

Transport Layer



Assist. Prof. Dr. İlktan AR

0.1 Course – Tentative Outline

Week	Date	Topic
1	28 Sept.	Computer Networks and the Internet
2	5 Oct.	Computer Networks and the Internet
3	12 Oct.	Application Layer
4	26 Oct.	Application Layer
5	2 Nov.	Transport Layer, Wireshark usage and examples
6	9 Nov.	**First Midterm Exam (15:00-17:00 Balat Salon & PC.Lab3)**
7	16 Nov.	Transport Layer
8	23 Nov.	The Network Layer: Data Plane
9	30 Nov.	The Network Layer: Control Plane
10	30 Nov.	The Link Layer and Local Area Networks
11	7 Dec.	**Second Midterm Exam (15:00-17:00 Balat Salon & PC.Lab3)**
12	14 Dec.	Security Technology and Principles
13	21 Dec.	Security Technology and Principles
14	28 Dec.	Security Technology and Principles, Talk for Final Exam
FN	???	**Final Exam**

0.2 Previous Week

- We will focus on chapters 3: Transport Layer.
- We will also focus on Wireshark usage and examples.
- Talk for First Midterm Exam.



Your duties:

✓ Review this lecture note via learn.khas.edu.tr

✓ Read chapter 3 from Kurose & Ross's textbook.

Week 5 - Agenda

01 Introduction and Transport-Layer Services

02 Multiplexing and Demultiplexing

03 Connectionless Transport: UDP

04 Principles of Reliable Data Transfer

05 Next Week

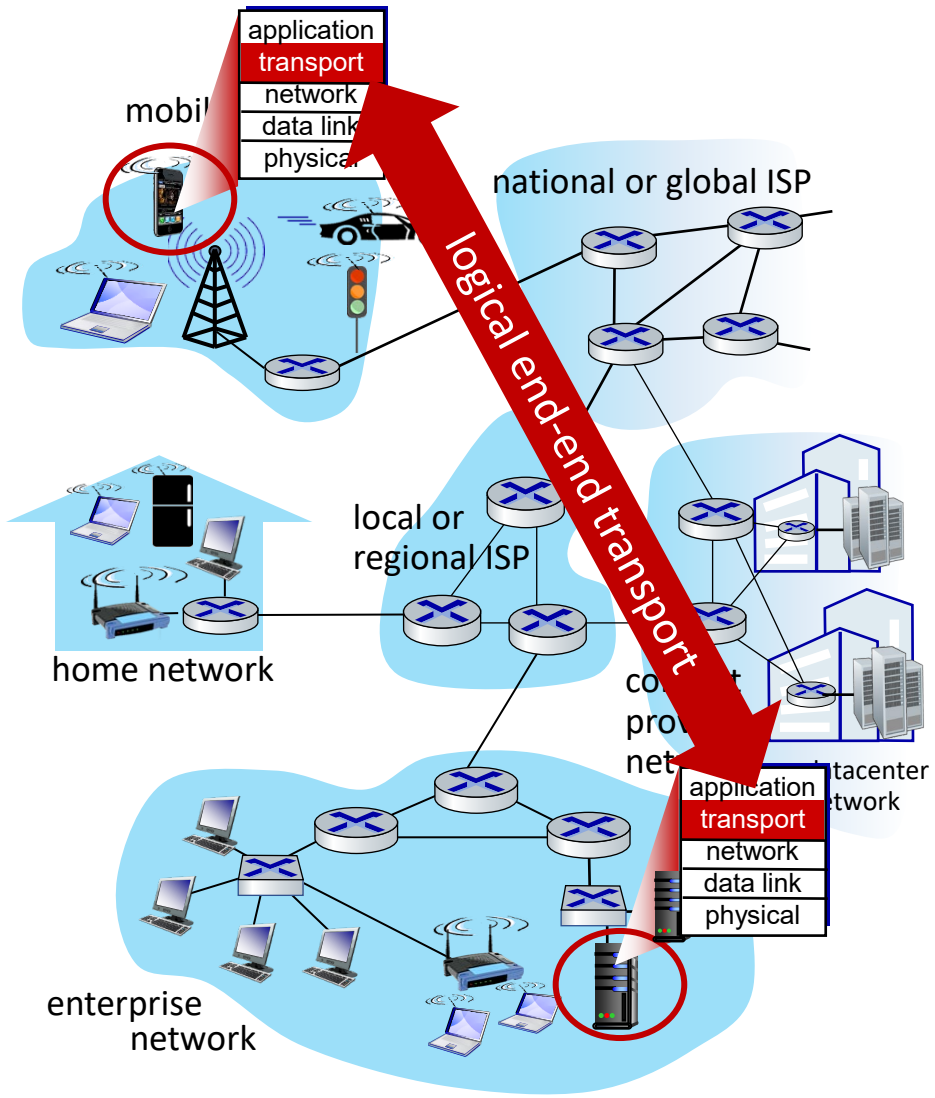
1.1 Introduction

- A transport-layer protocol provides for **logical communication** between application processes running on different hosts.

- By logical communication, we mean that from an application's perspective, it is as if the hosts running the processes were directly connected; in reality, the hosts may be on opposite sides of the planet, connected via numerous routers and a wide range of link types.

- Application processes use the logical communication provided by the transport layer to send messages to each other.

1.2 Introduction (cont.)



On the sending side (the transport layer):

- Receive a message from the application,
- Split it into chunks if needed; add a transport header,
- Passes each segment to the network layer,

The Internet has two protocols: TCP and UDP. Each of these protocols offers different service trade-offs.

1.3 Relationship Between Transport and Network Layers

- network layer: logical communication between *hosts*
- transport layer: logical communication between *processes*
  - relies on, enhances, network layer services

household analogy:

12 kids in Ann's house sending letters to 12 kids in Bill's house:

- hosts = houses
- processes = kids
- app messages = letters in envelopes
- transport protocol = Ann and Bill who demux to in-house siblings
- network-layer protocol = postal service

1.4 Overview of the Transport Layer on the Internet

- Recall that the Internet makes two distinct transport-layer protocols available to the application layer.

TCP (Transmission Control Protocol)

- Reliable, ordered byte stream; connection-oriented.
- Retransmission and congestion control are built in.

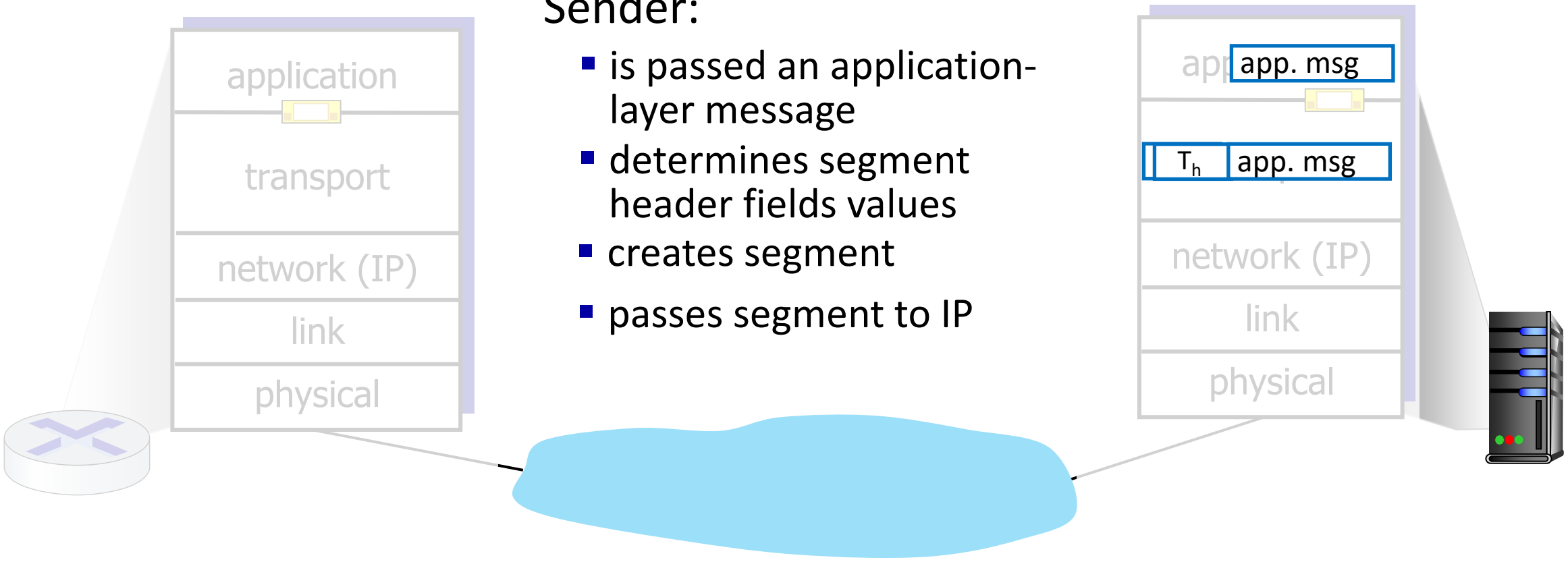
UDP (User Datagram Protocol)

- Connectionless datagrams with a small header.
- No built-in delivery or ordering guarantee.

- Applications choose based on their requirements.

1.5 Transport Layer Actions

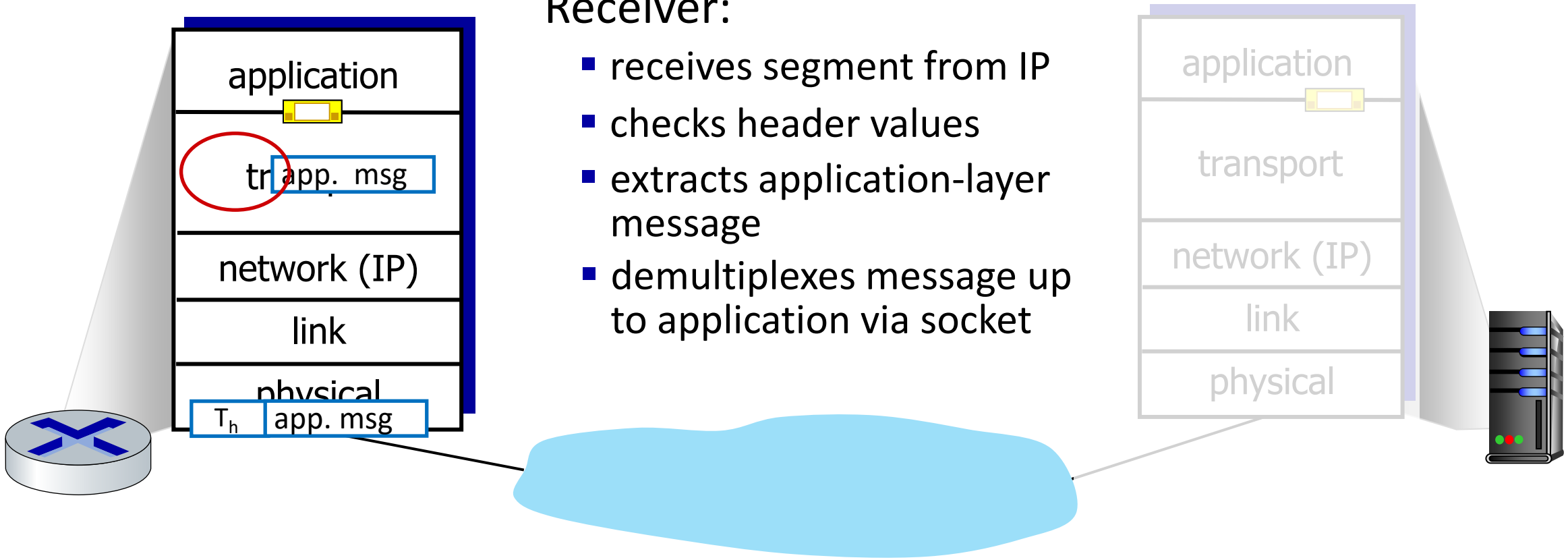
- Transport extends IP delivery from one host to a specific application process.
- **Multiplexing** gathers data from sockets at the sender; **demultiplexing** delivers it to the right socket at the receiver.



Sender:

- is passed an application-layer message
- determines segment header fields values
- creates segment
- passes segment to IP

1.6 Transport Layer Actions (cont.)



Receiver:

- receives segment from IP
- checks header values
- extracts application-layer message
- demultiplexes message up to application via socket

Week 5 - Agenda

01 Introduction and Transport-Layer Services

02 Multiplexing and Demultiplexing

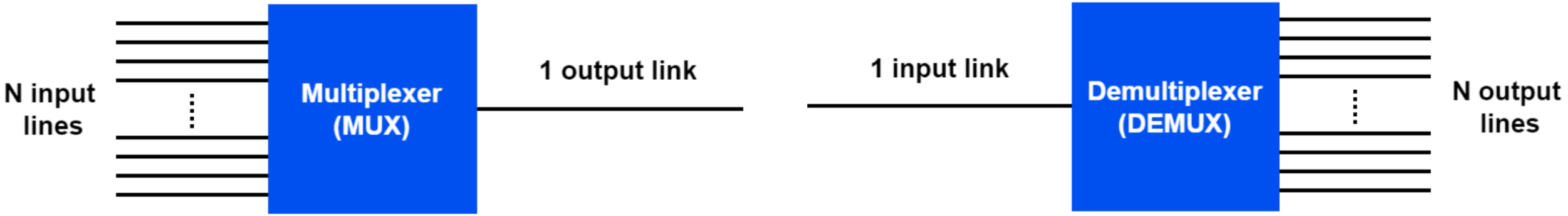
03 Connectionless Transport: UDP

04 Principles of Reliable Data Transfer

05 Next Week

2.1 Multiplexing and Demultiplexing

- The transport layer has the responsibility of delivering the data in segments to the appropriate application process running in the host.
- One laptop may run a browser, a video call, a game, and a file transfer at once.
- Incoming segments must reach the correct application socket.
- Transport-layer header fields, especially port numbers, help identify the destination.



2.2 Multiplexing and Demultiplexing (cont.)

- **Multiplexing (sender)**: gather data from multiple sockets, add transport headers, and pass segments to IP.

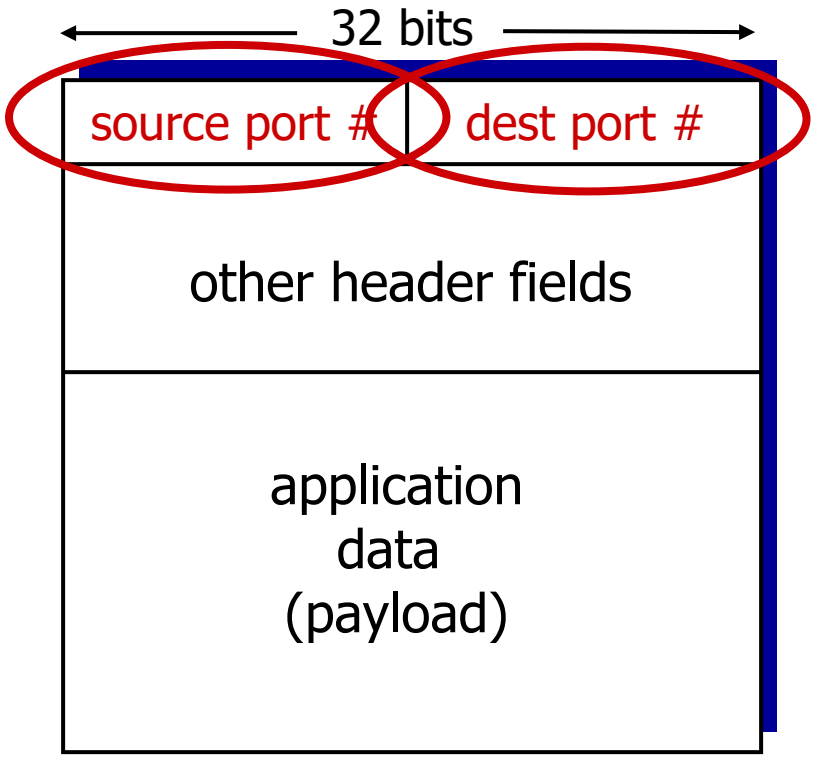
- **Demultiplexing (receiver)**: inspect header fields and deliver each segment to the correct socket.

2.3 Multiplexing and Demultiplexing (cont.)

- Transport-layer multiplexing requires

(1) that sockets have unique identifiers, and

(2) that each segment have special fields that indicate the socket to which the segment is to be delivered. These special fields, illustrated in Fig., are the source port number field and the destination port number field.



TCP/UDP segment format

• Port numbers are 16-bit values: 0–65535. They help the operating system identify the destination application or service. Common examples: HTTP 80, HTTPS 443, DNS 53.

- The port numbers ranging from 0 to 1023 are called **well-known port numbers;** applications also use other ports.

2.4 Ports in Practice: One Host, Many Connections

CLIENT PROCESS

Browser on a laptop

192.0.2.10:53124



WEB SERVER

HTTPS service

203.0.113.20:443

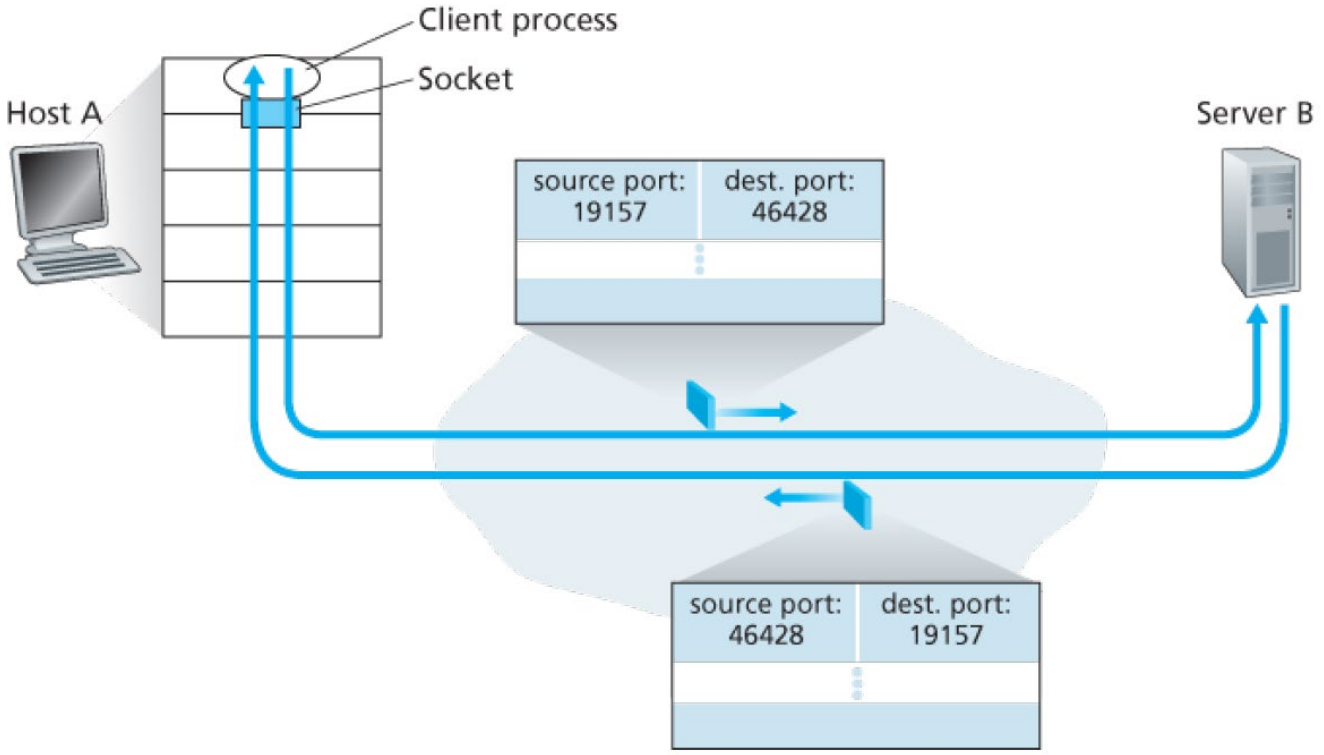
A TCP connection is identified by four values:

SOURCE IP	SOURCE PORT	DESTINATION IP	DESTINATION PORT
192.0.2.10	53124	203.0.113.20	443

Why it matters: port information helps the OS deliver incoming data to the correct socket, not just the correct host.

2.5 Connectionless Multiplexing and Demultiplexing

- A UDP socket is fully identified by a two-tuple consisting of a destination IP address and a destination port number.
- Therefore, if two UDP segments have different source IP addresses and/or source port numbers, but have the same *destination* IP address and *destination* port number, then the two segments will be directed to the same destination process via the same destination socket.



2.6 Connection-Oriented Multiplexing and Demultiplexing

- TCP socket identified by 4 tuple:

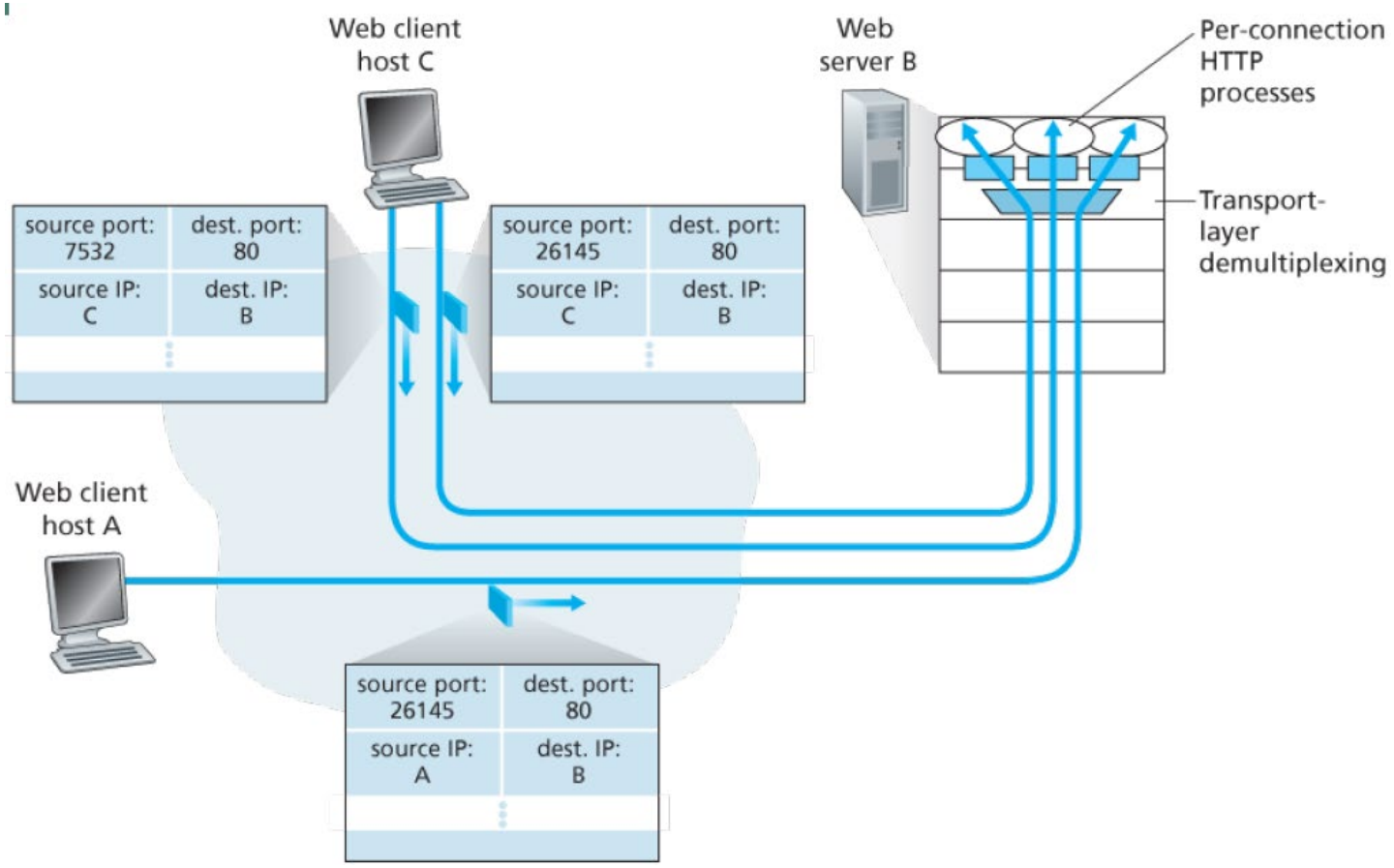
a. Source IP address

b. Source port number

c. Destination IP address

d. Destination port number

- Changing either endpoint distinguishes a different TCP connection.



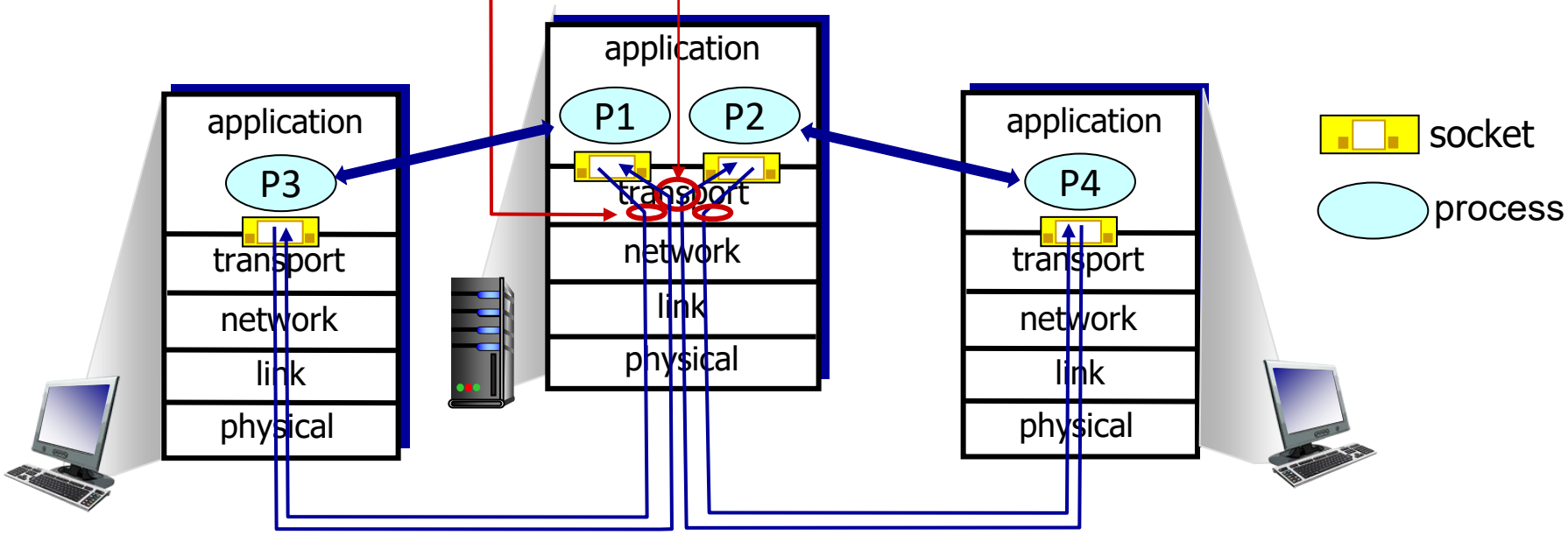
2.7 Multiplexing and Demultiplexing

multiplexing as sender:

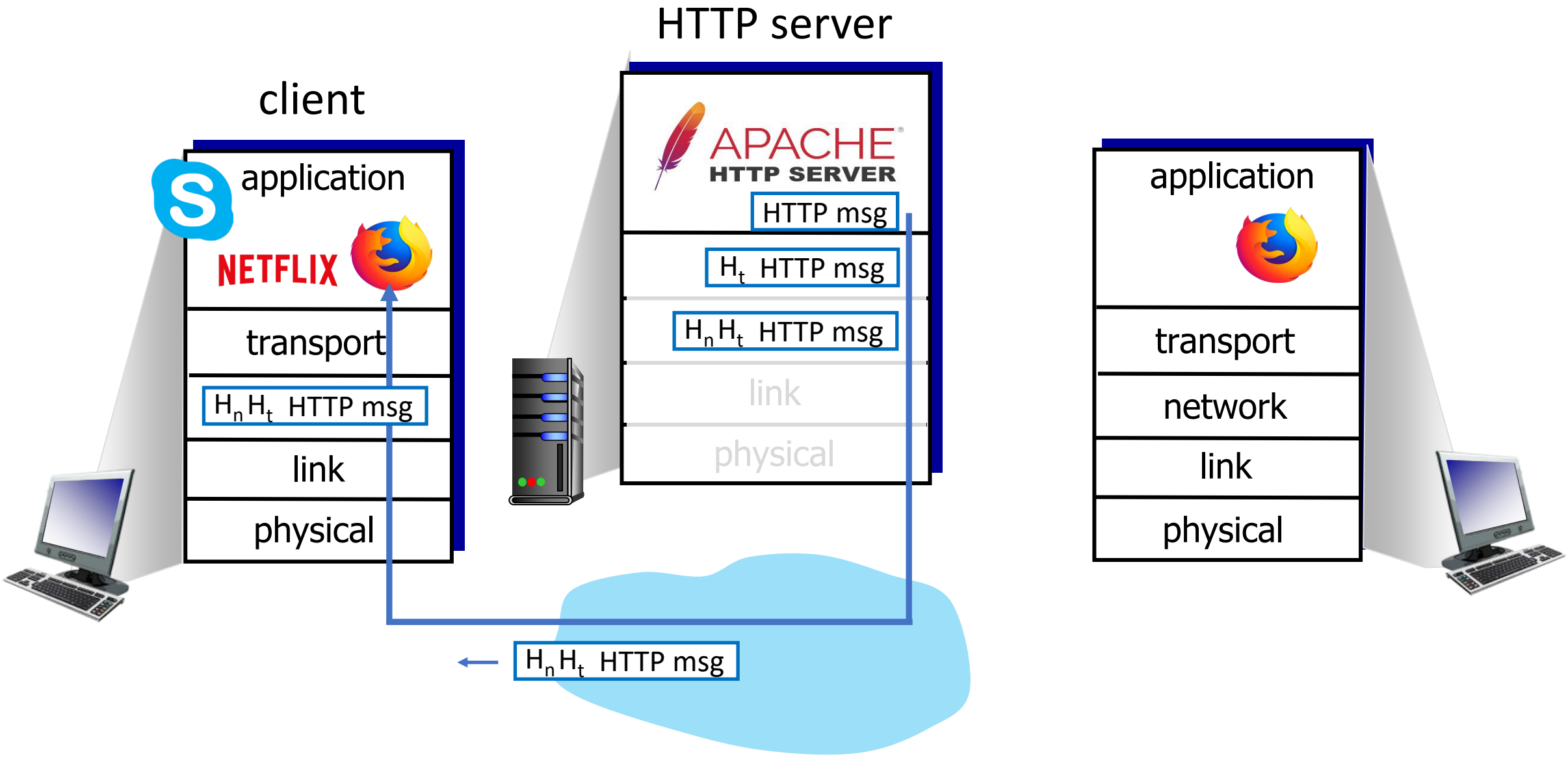
handle data from multiple sockets, add transport header (later used for demultiplexing)

demultiplexing as receiver:

use header info to deliver received segments to correct socket

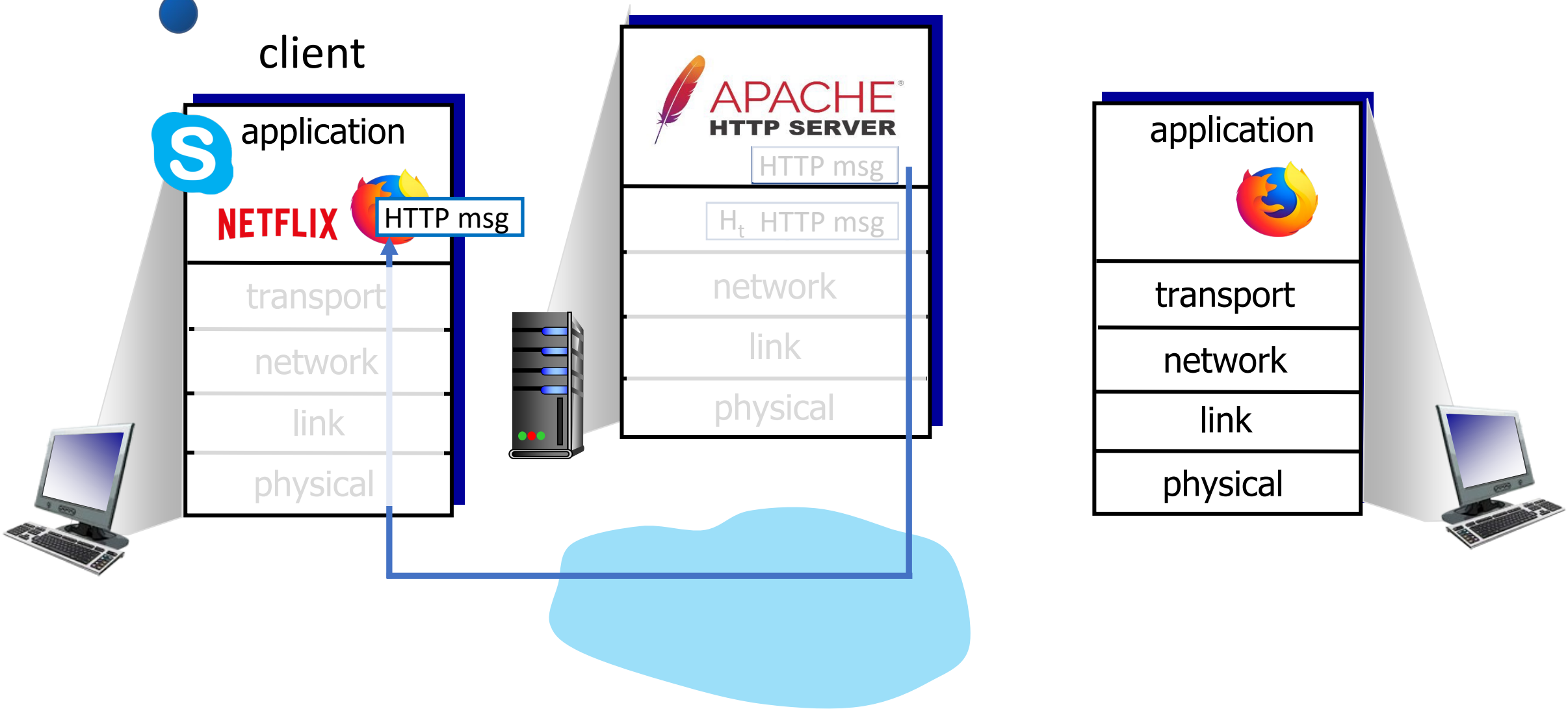


2.8 Multiplexing and Demultiplexing (cont.)

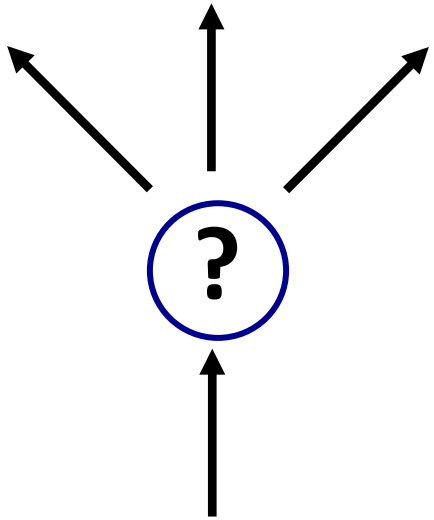


2.9 Multiplexing and Demultiplexing (cont.)

Q: Your browser, a video-call app, and a game are all active. How does incoming data reach the right process?

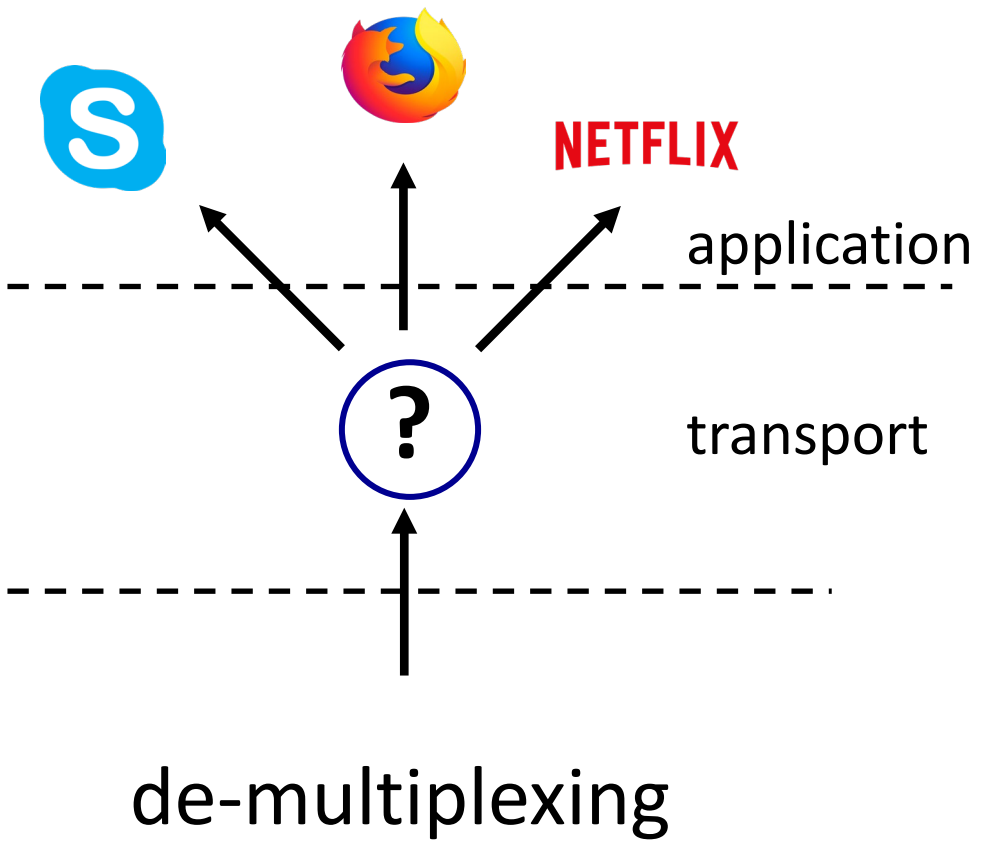


2.10 Multiplexing and Demultiplexing (cont.)



de-multiplexing

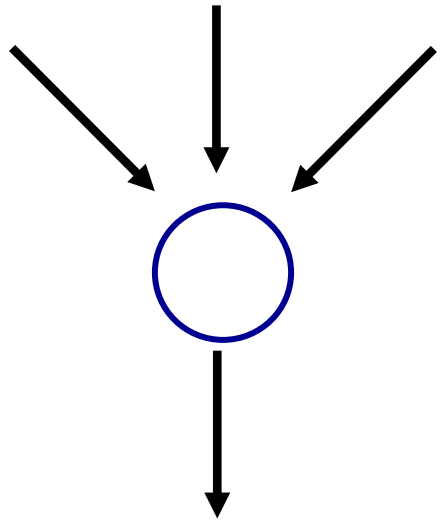
2.11 Multiplexing and Demultiplexing (cont.)



2.12 Multiplexing and Demultiplexing (cont.)

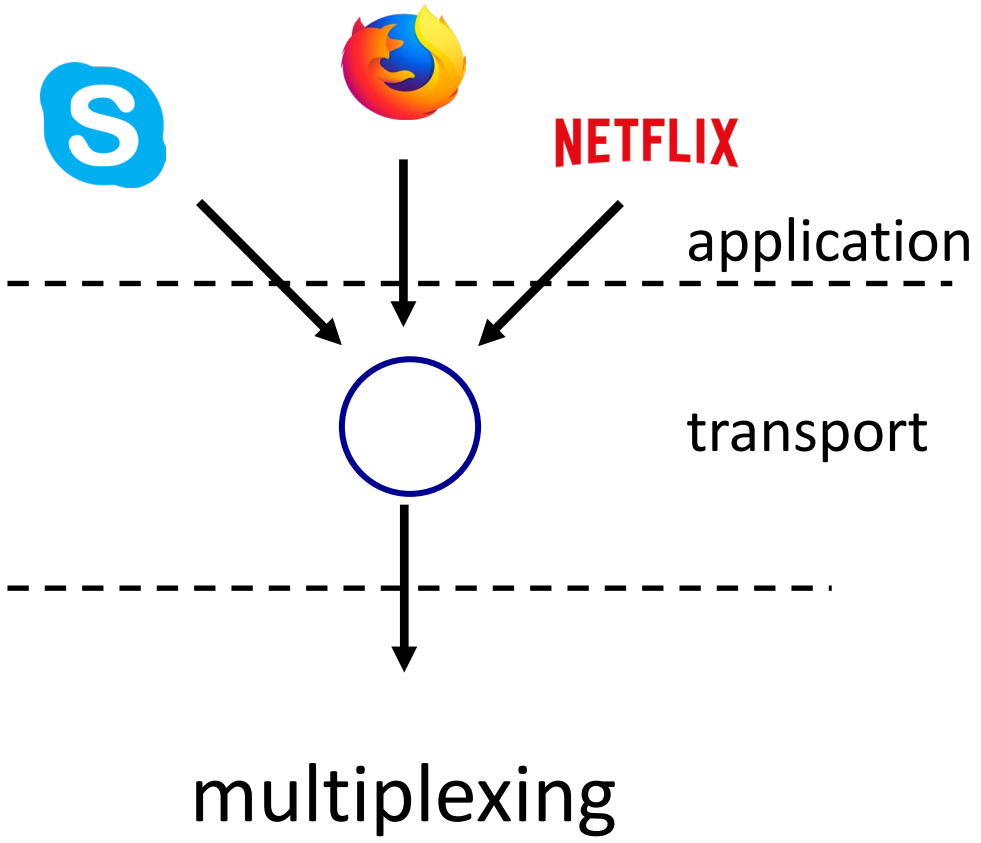


2.13 Multiplexing and Demultiplexing (cont.)



multiplexing

2.14 Multiplexing and Demultiplexing (cont.)



2.15 Multiplexing and Demultiplexing (cont.)



Week 5 - Agenda

01 Introduction and Transport-Layer Services

02 Multiplexing and Demultiplexing

03 Connectionless Transport: UDP

04 Principles of Reliable Data Transfer

05 Next Week

3.1 Connectionless Transport: UDP

- UDP, defined in [RFC 768], does just about as little as a transport protocol can do.
- UDP adds a small header and checksum, then hands the segment to IP.
- In fact, if the application developer chooses UDP instead of TCP, then the application is almost directly talking with IP.
- **With UDP there is no handshaking between sending and receiving transport-layer entities** before sending a segment. For this reason, UDP is said to be *connectionless*.
- It provides no built-in guarantee of delivery, ordering, or retransmission.

3.2 UDP: User Datagram Protocol [RFC 768]

INTERNET STANDARD

RFC 768

J. Postel
ISI
28 August 1980

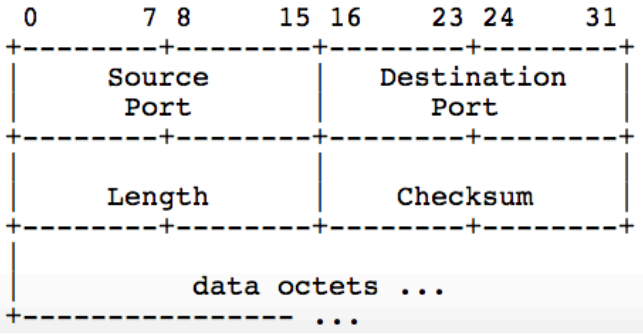
User Datagram Protocol

Introduction

This User Datagram Protocol (UDP) is defined to make available a datagram mode of packet-switched computer communication in the environment of an interconnected set of computer networks. This protocol assumes that the Internet Protocol (IP) [1] is used as the underlying protocol.

This protocol provides a procedure for application programs to send messages to other programs with a minimum of protocol mechanism. The protocol is transaction oriented, and delivery and duplicate protection are not guaranteed. Applications requiring ordered reliable delivery of streams of data should use the Transmission Control Protocol (TCP) [2].

Format



3.3 UDP: User Datagram Protocol

UDP is a best-effort datagram service

- Packets can be lost.
- Packets can arrive out of order.
- There is no connection setup.

Why use UDP?

- Small header: 8 bytes.
- No transport-level retransmission delay.
- Useful when the application can manage loss or timing itself.

Why is there a UDP?

- no connection establishment (which can add RTT delay)
- simple: no connection state at sender, receiver
- small header size
- no congestion control
  - UDP can blast away as fast as desired!
  - can function in the face of congestion

3.4 UDP Today: Low Latency and the QUIC Twist

REAL-TIME APPS

- Fresh updates can matter more than retransmitting stale data.
- Many real-time applications use UDP-based transport when low delay matters.
- The application can decide how to handle loss.

DNS LOOKUPS

- Queries and replies are often small.
- UDP keeps the common exchange lightweight.
- TCP is also used when required, including some larger or fallback exchanges.

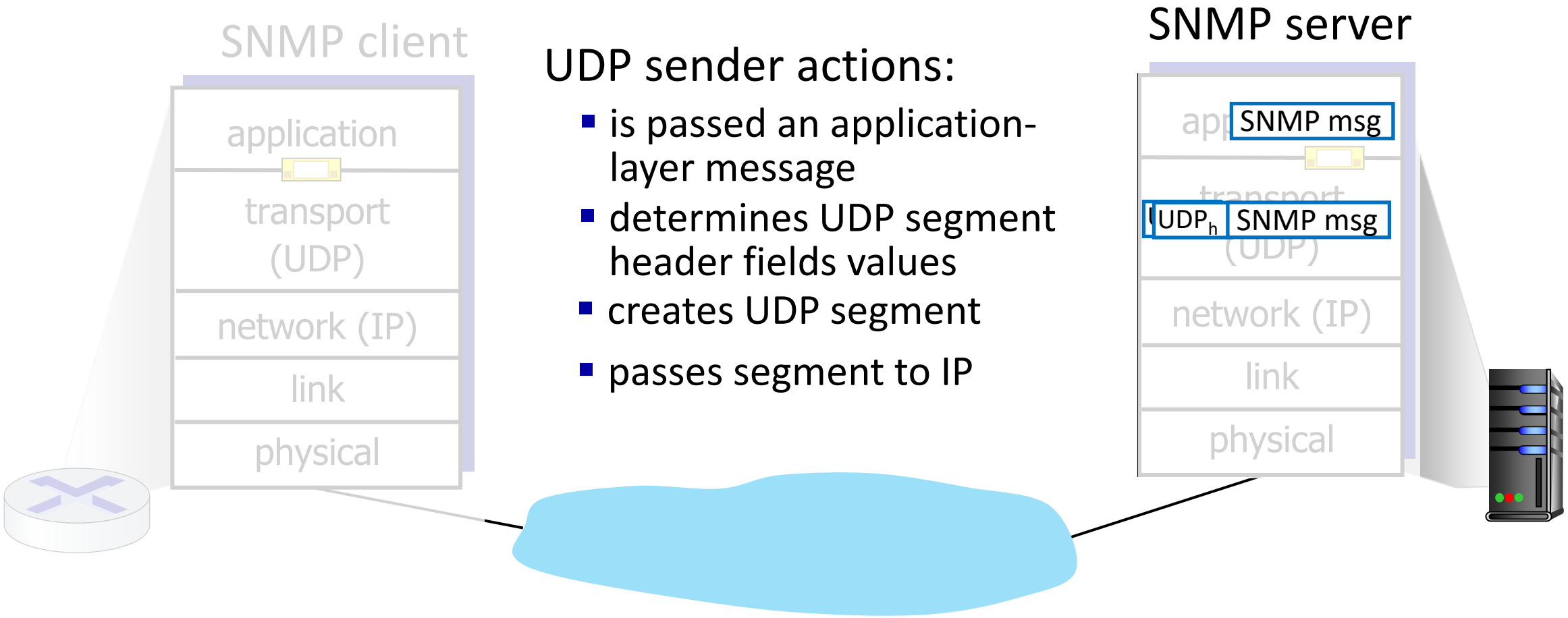
HTTP/3 + QUIC

- Modern web traffic can use QUIC over UDP.
- QUIC adds encryption, streams, congestion control, and reliable delivery.
- UDP itself still does not guarantee delivery.

Key distinction: QUIC uses UDP as its substrate, then implements reliability and security above UDP.

QUIC (Quick UDP Internet Connections) is a transport-layer protocol that runs over UDP. HTTP/3 is an application-layer protocol that uses QUIC.

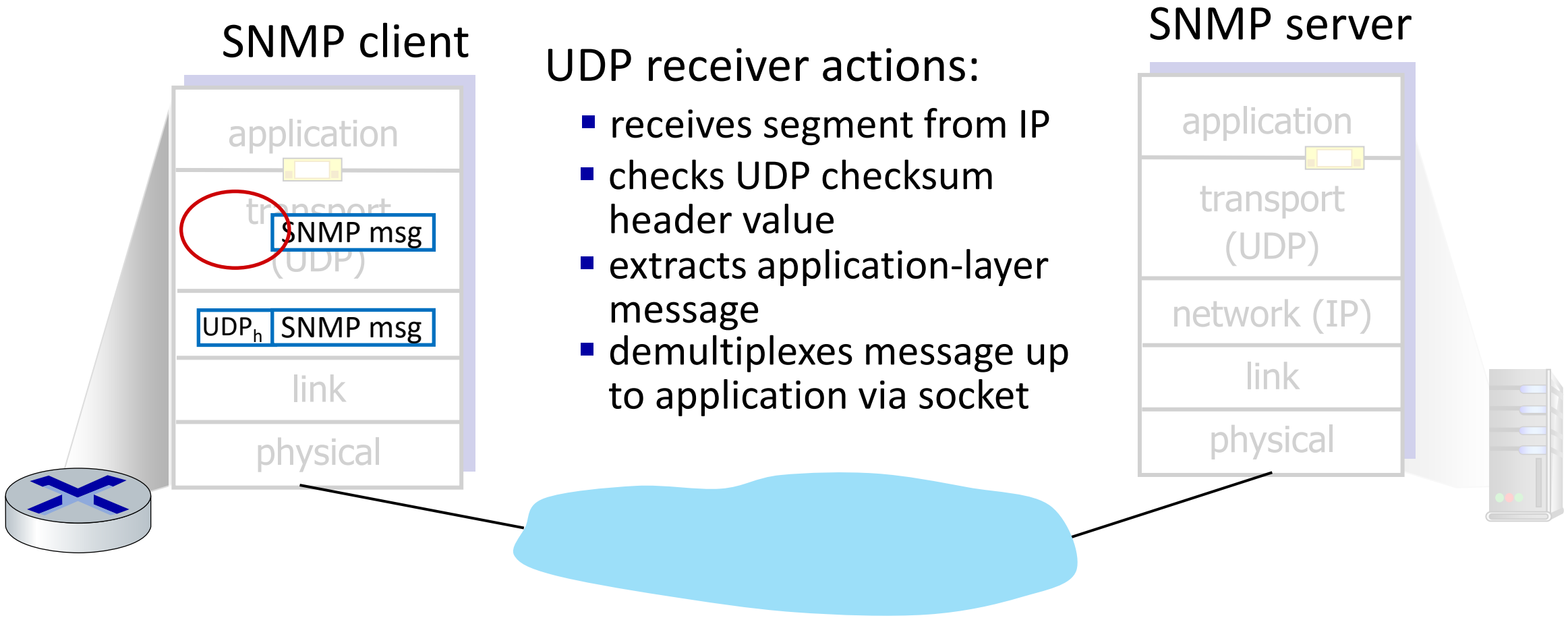
3.5 UDP: Transport Layer Actions



UDP sender actions:

- is passed an application-layer message
- determines UDP segment header fields values
- creates UDP segment
- passes segment to IP

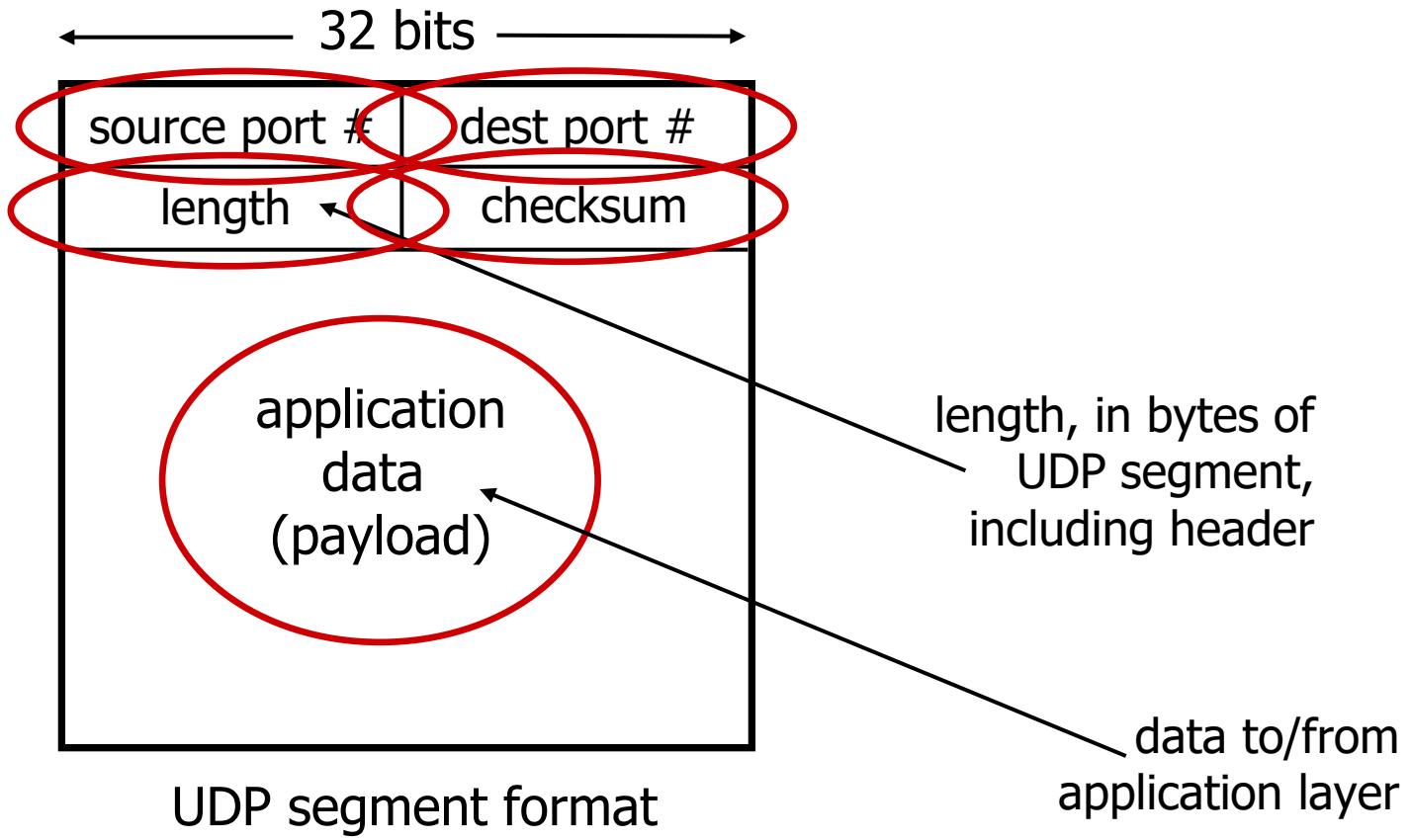
3.6 UDP: Transport Layer Actions (cont.)



UDP receiver actions:

- receives segment from IP
- checks UDP checksum header value
- extracts application-layer message
- demultiplexes message up to application via socket

3.7 UDP Segment Header

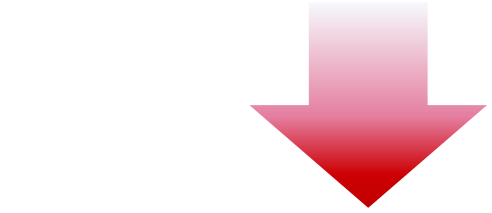


The TCP segment has 20 bytes of header overhead in every segment, whereas UDP has only 8 bytes of overhead.

3.8 UDP Checksum

Goal: detect errors (*i.e.*, flipped bits) in transmitted segment

	1st number	2nd number	sum
Transmitted:	5	6	11
Received:	4	6	11



receiver-computed checksum ≠ sender-computed checksum (as received)



3.9 UDP Checksum

Goal: detect errors (*i.e.,* flipped bits) in transmitted segment

sender:

- Treat the segment contents as 16-bit words.
- Compute a one's-complement checksum.
- Put the result in the checksum field.

receiver:

- Recompute the checksum.
- A mismatch indicates an error.
- A match does not prove that every possible error is absent.

Checksum detects many errors; it does not repair them or provide encryption.


10 mins break,
QR attendance!
TIME
FOR A
BREAK!

Week 5 - Agenda

01 Introduction and Transport-Layer Services

02 Multiplexing and Demultiplexing

03 Connectionless Transport: UDP

04 Principles of Reliable Data Transfer

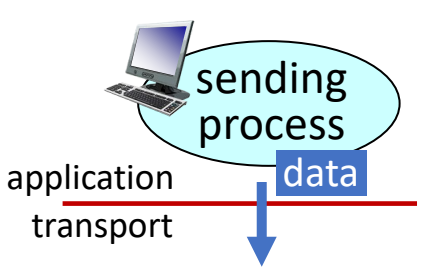
05 Next Week

4.1 Principles of Reliable Data Transfer

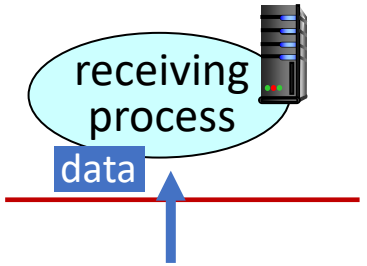
- First consider the simplest case, in which the underlying channel is completely reliable (left fig.).
- Complexity of reliable data transfer protocol will depend (right fig.) on characteristics of unreliable channel (lose, corrupt, reorder data?)



reliable service *abstraction*



sender-side of reliable data transfer protocol



receiver-side of reliable data transfer protocol



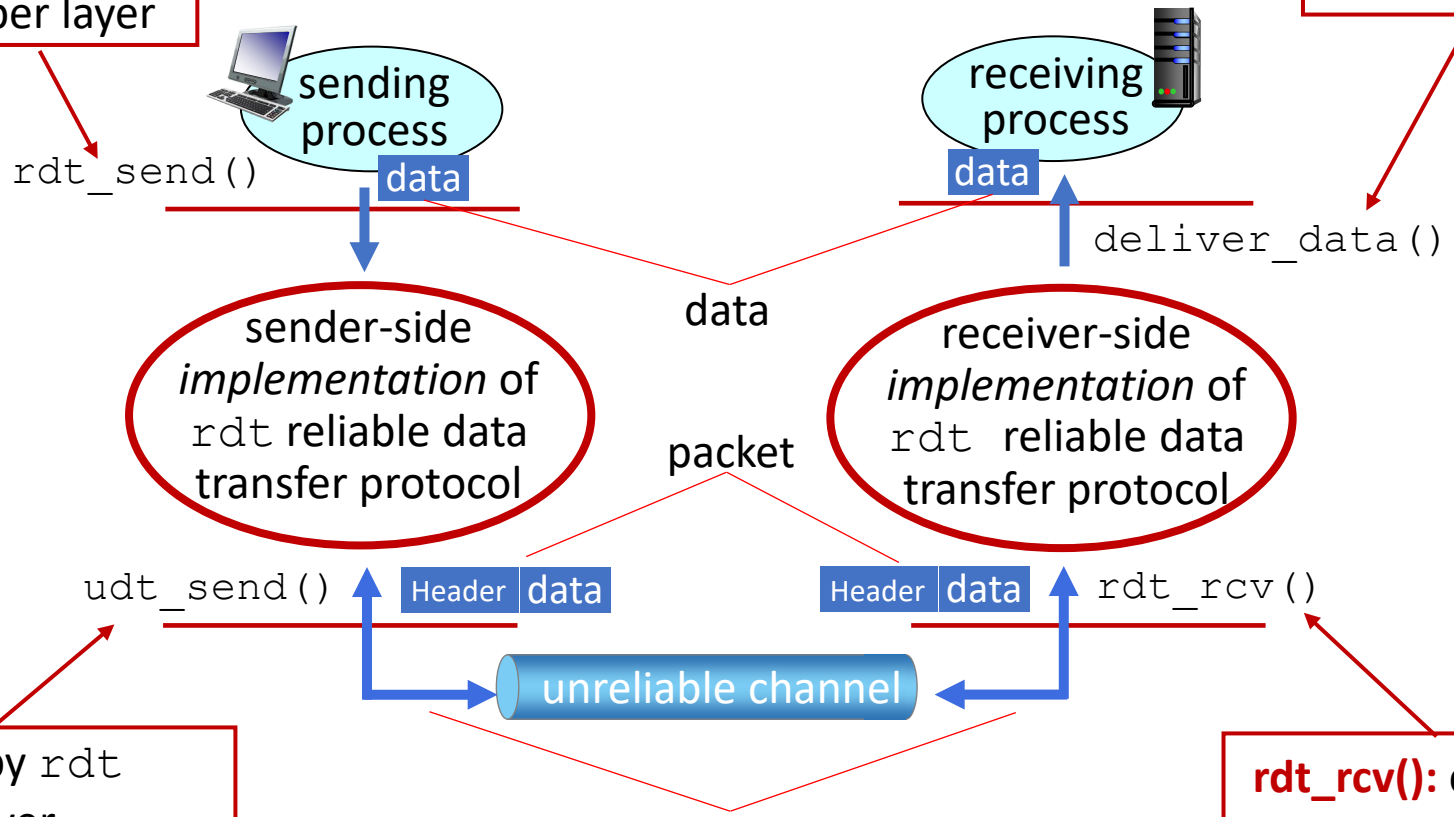
reliable service *implementation*

RDT adds mechanisms to recover from failures.

4.2 Reliable Data Transfer Protocol (rdt): Interfaces

rdt_send(): called from above, (e.g., by app.). Passed data to deliver to receiver upper layer

deliver_data(): called by rdt to deliver data to upper layer



udt_send(): called by rdt to transfer packet over unreliable channel to receiver

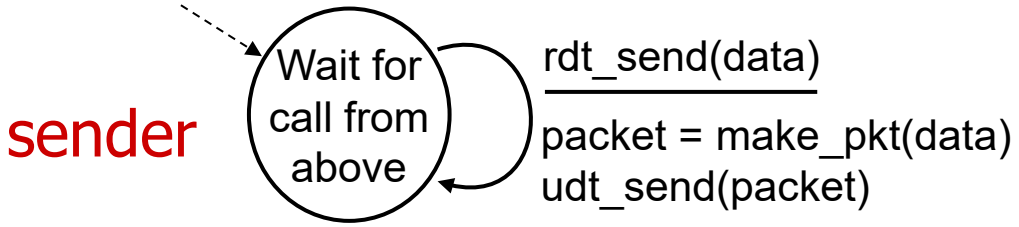
rdt_rcv(): called when packet arrives on receiver side of channel

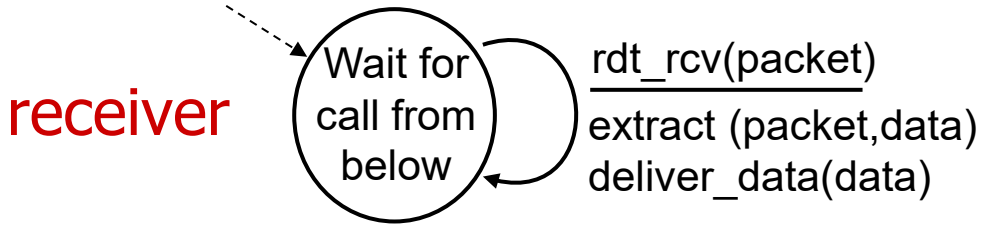
4.3 Rdt1.0: Reliable Transfer over a Reliable Channel

- underlying channel perfectly reliable
  - no bit errors
  - no loss of packets
- *Rdt1.0 assumes a perfect channel:*

send → receive, with no recovery logic needed.







The simplest case possible: The sender just sends and it pops out the other side (perhaps after some delay) perfectly.

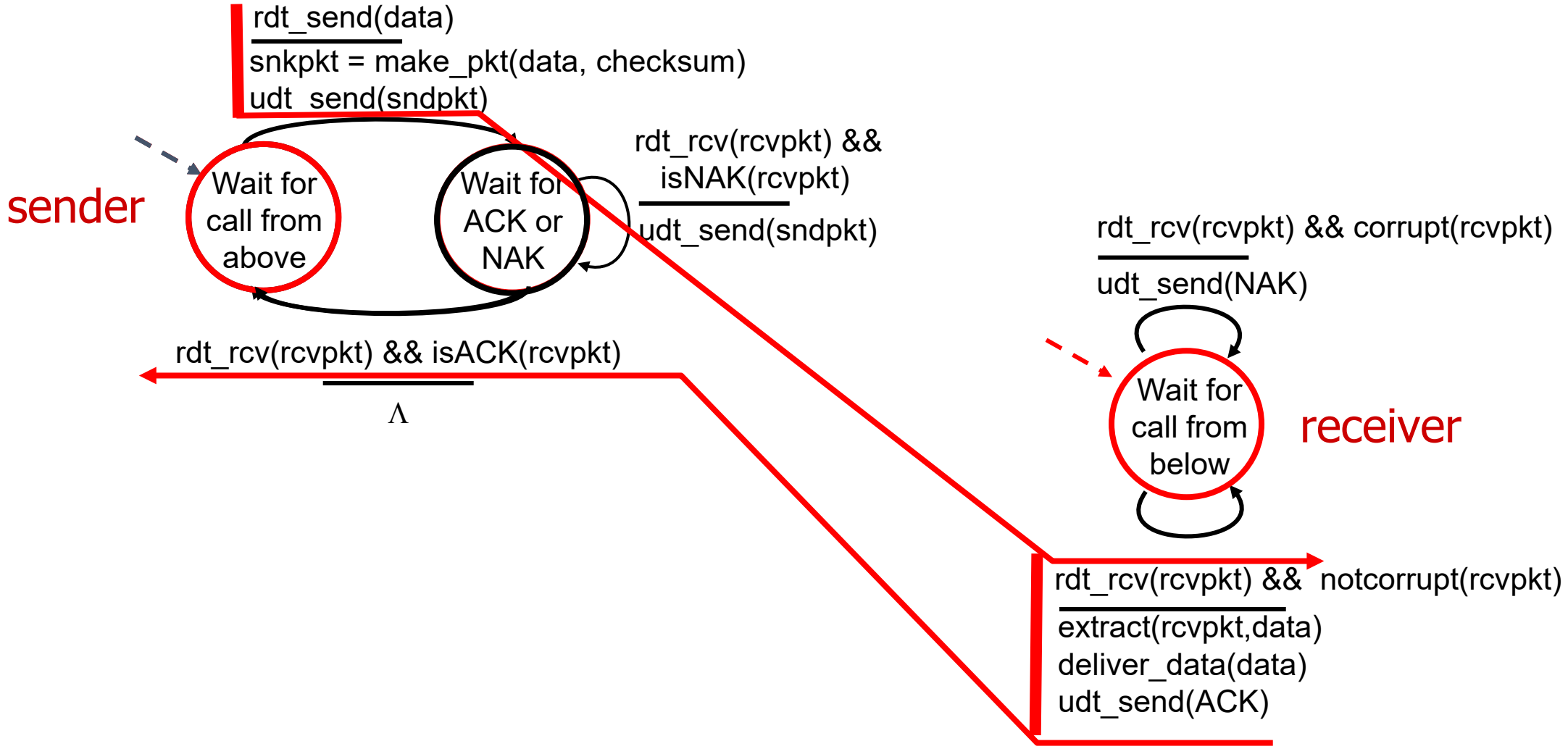
4.4 Rdt2.0: Channel with Bit Errors

- The channel can corrupt bits.
- A checksum detects many errors but does not repair them.
- *How can the sender learn what happened?*
  - *acknowledgements (ACKs):* receiver reports that a packet arrived intact.
  - *negative acknowledgements (NAKs):* receiver reports an error.
  - sender *retransmits* pkt on receipt of NAK

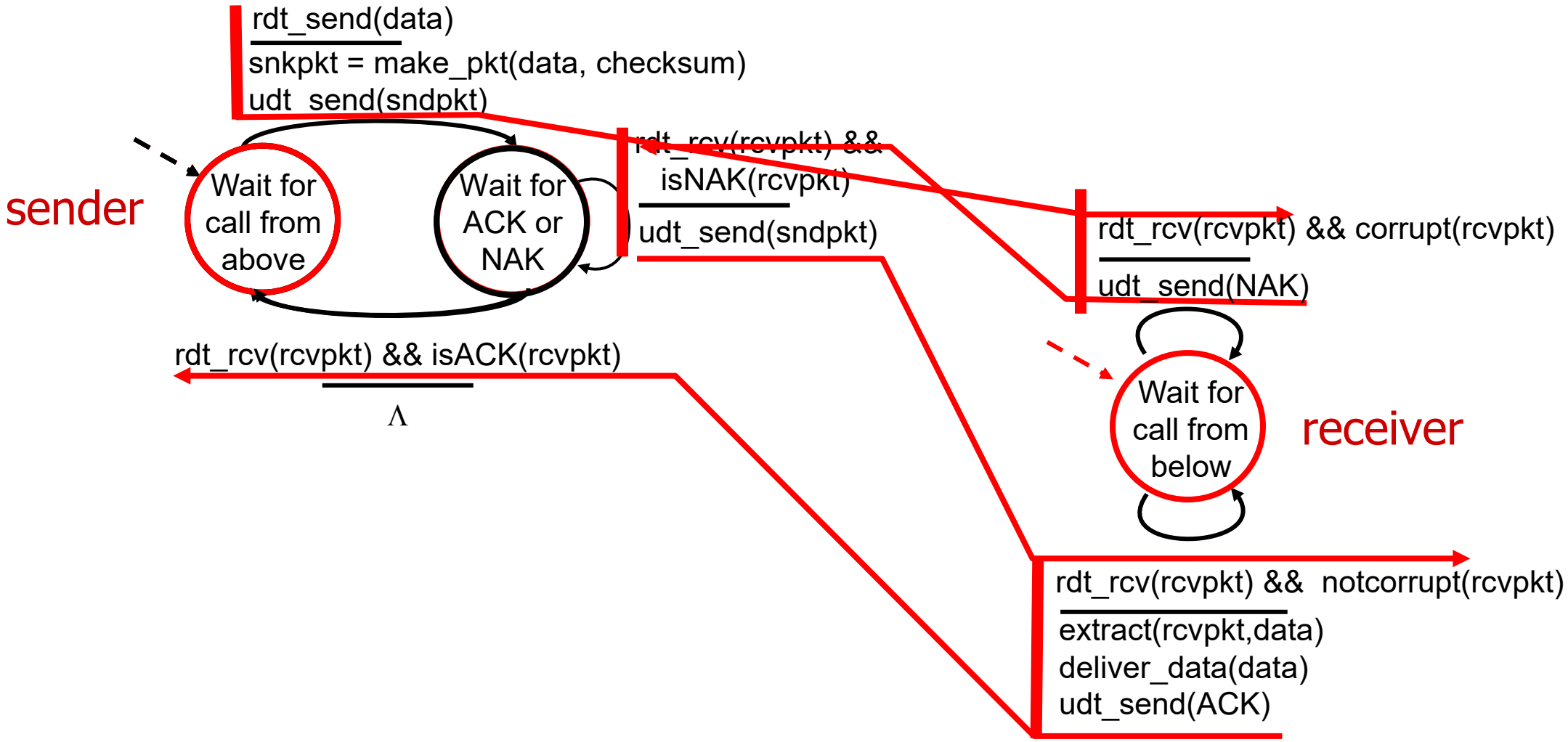
stop and wait

sender sends one packet, then waits for receiver response

4.5 Rdt2.0: Operation with No Errors



4.6 Rdt2.0: Corrupted Packet Scenario



4.7 Rdt2.0 has a Fatal Flaw!

Problem: an ACK or NAK can also be corrupted.

The sender may not know whether the receiver got the data.

Retransmitting blindly can create duplicates.

Fix: add a **sequence number**.

The receiver uses it to recognize and discard duplicate packets.

stop and wait

sender sends one packet, then waits for receiver response

4.8 Reliable Data Transfer (rdt 3.0) ...

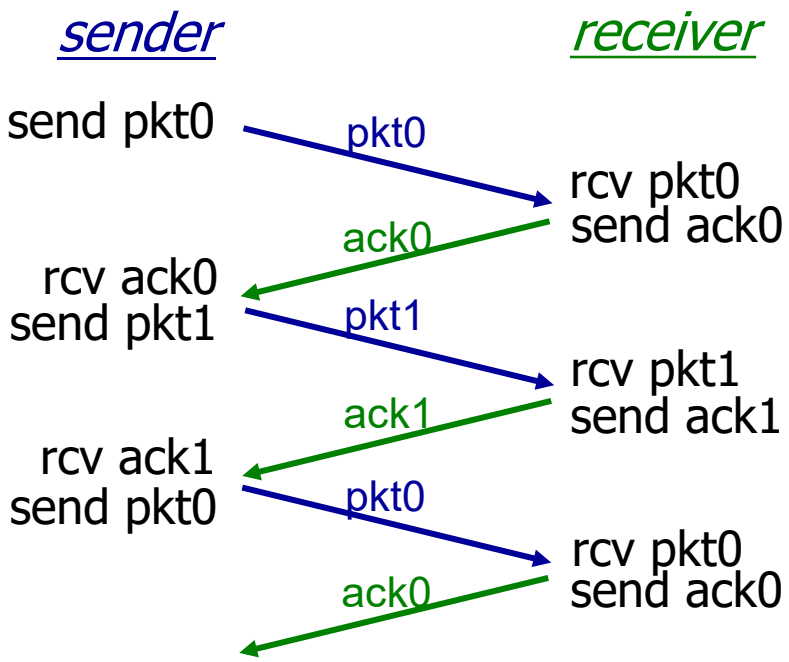
Rdt3.0 adds two mechanisms:

- Sequence numbers distinguish new data from duplicates.
- A timeout triggers retransmission if an ACK does not arrive.

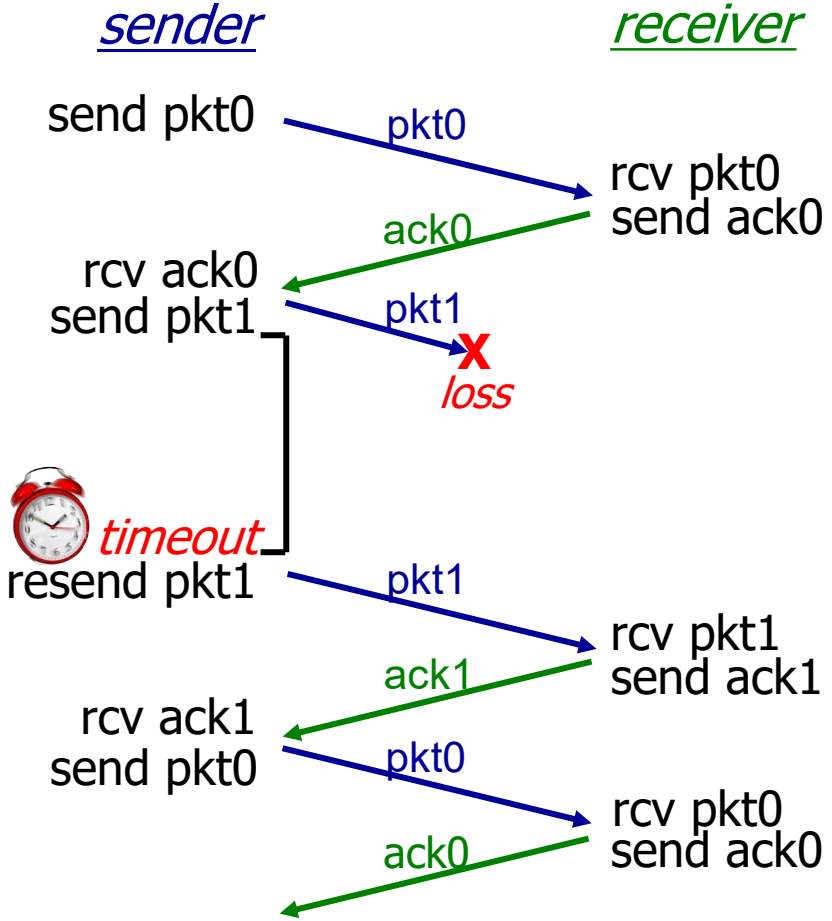
If an ACK is lost or delayed, the sender retransmits after timeout.

The receiver checks the sequence number and acknowledges duplicates without delivering data twice.

4.9 Rdt3.0 in Action

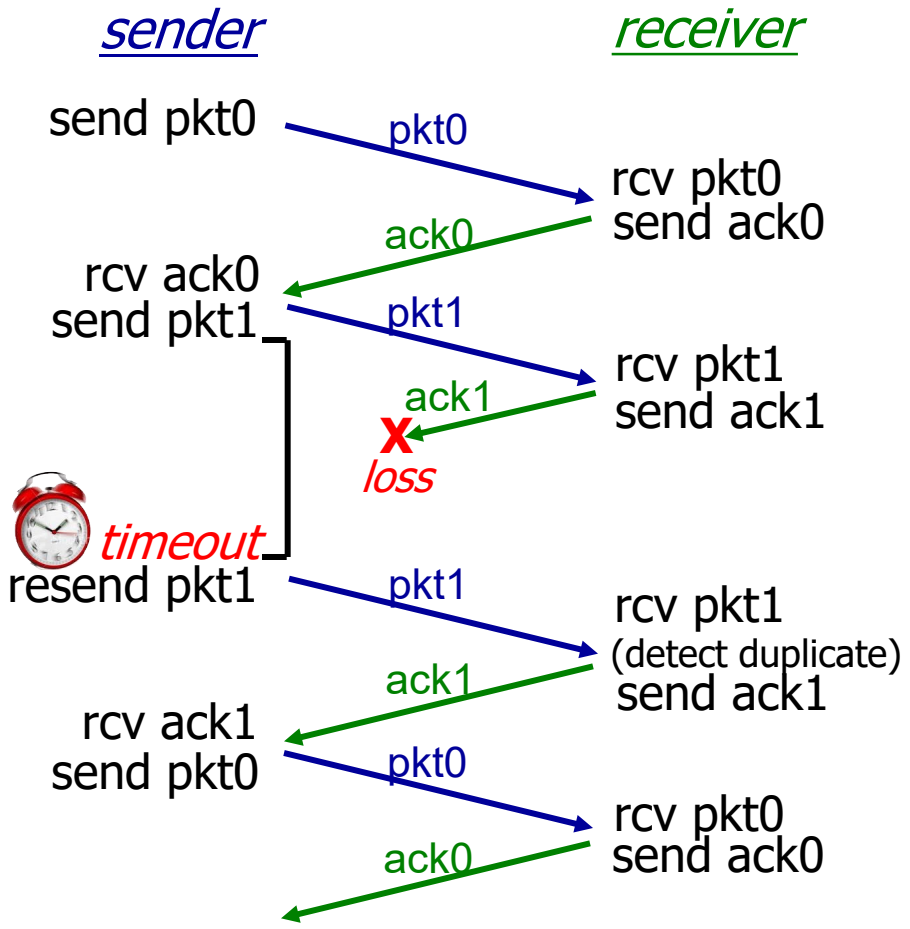


(a) no loss

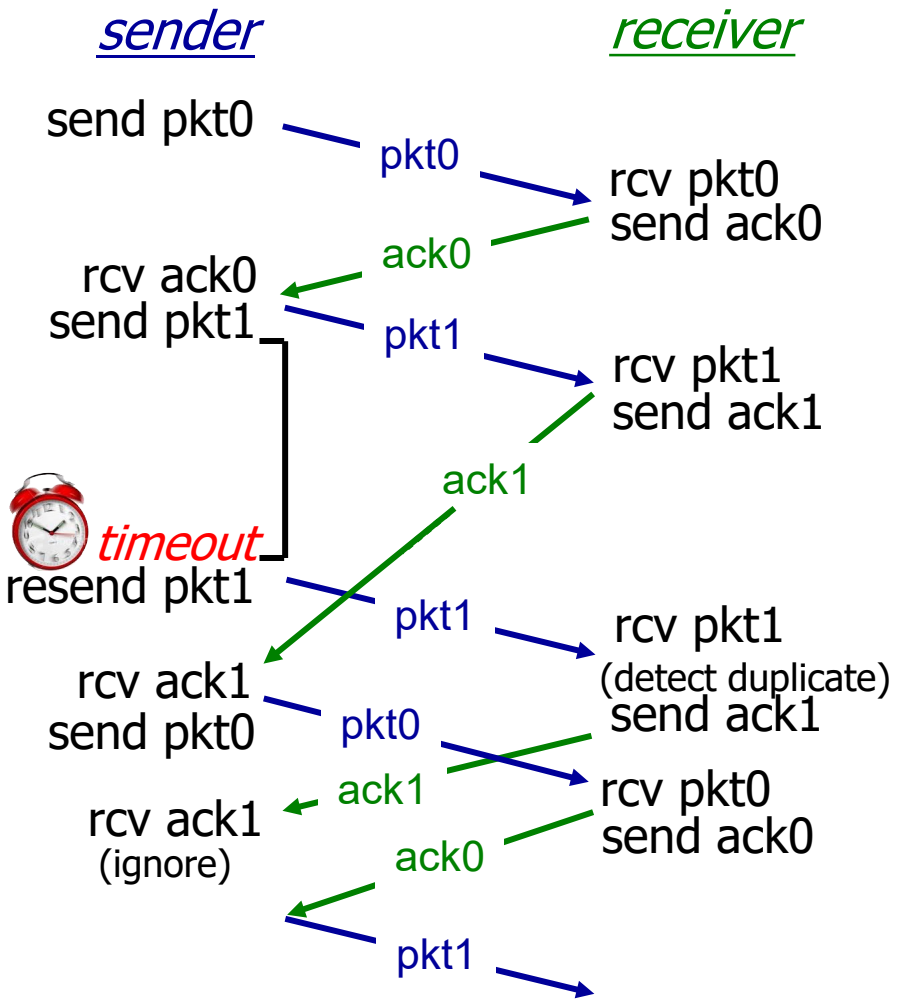


(b) packet loss

4.10 Rdt3.0 in Action (cont.)



(c) ACK loss



(d) premature timeout/ delayed ACK

4.11 Why Stop-and-Wait Wastes Capacity

Assume one 1 KB payload and an 80 ms round-trip time (RTT). Ignore headers and processing time.

1. SEND

Transmit 1 KB

2. WAIT

Wait for the ACK

3. ACK

RTT ≈ 80 ms

4. REPEAT

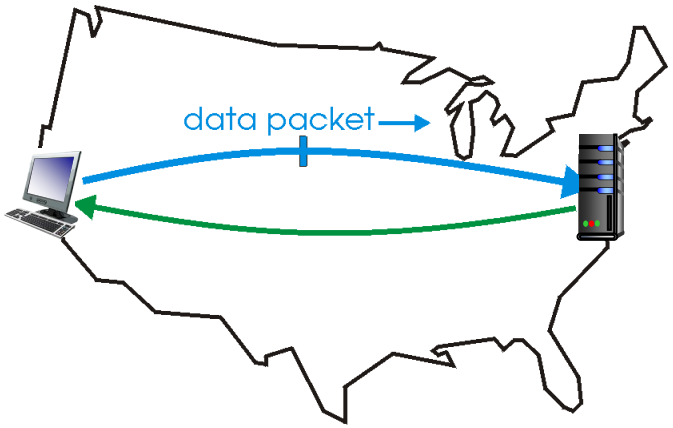
Only then send next

Throughput ≈ 1 KB / 0.08 s = 12.5 KB/s ≈ 100 kb/s

Pipelining sends several packets before waiting for all ACKs. It keeps the path busy while earlier packets are in flight.

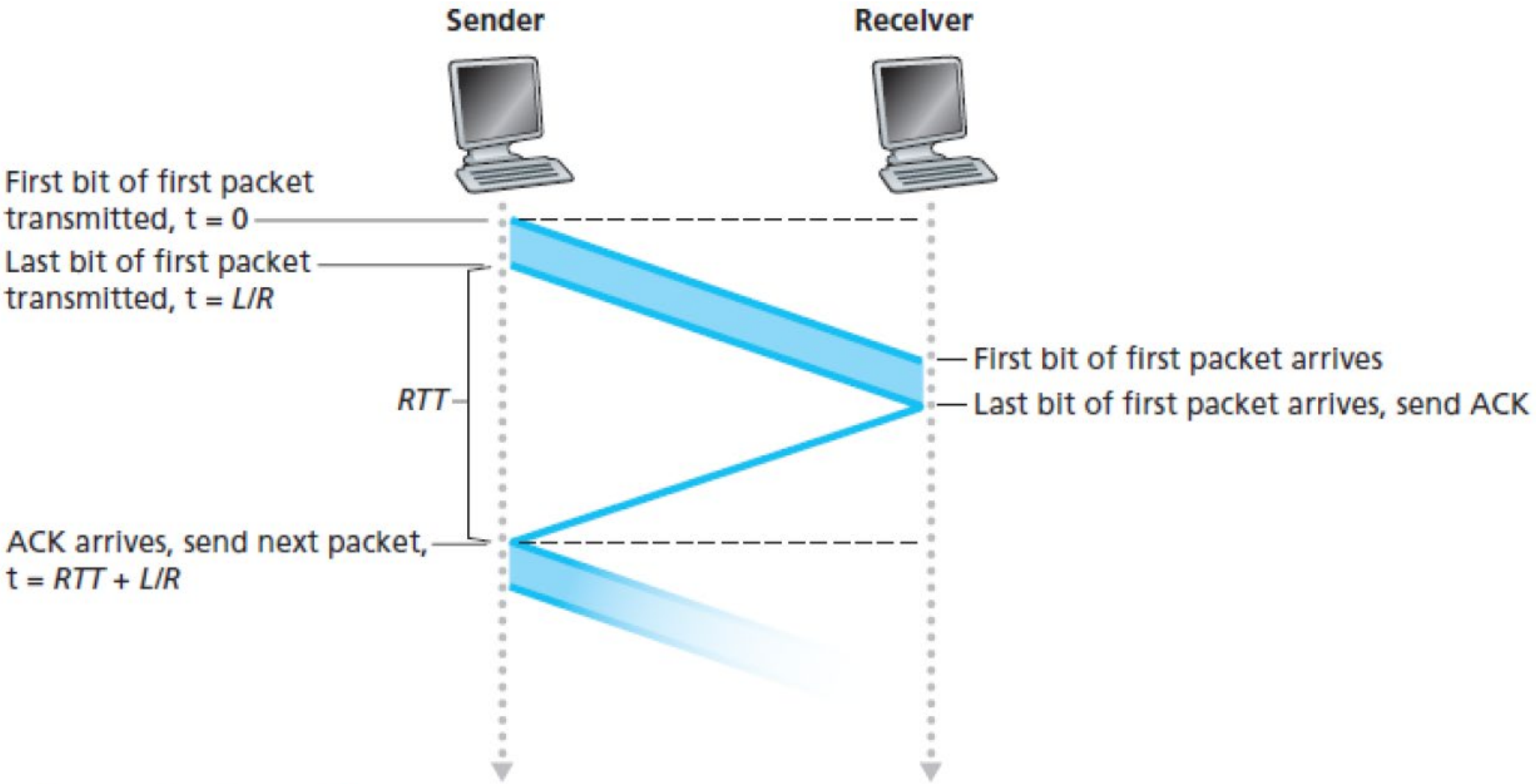
4.12 Enhance Rdt3.0 for Pipelines

- Pipelining lets the sender transmit multiple packets before their ACKs arrive.
- It requires a larger sequence-number space and buffering.



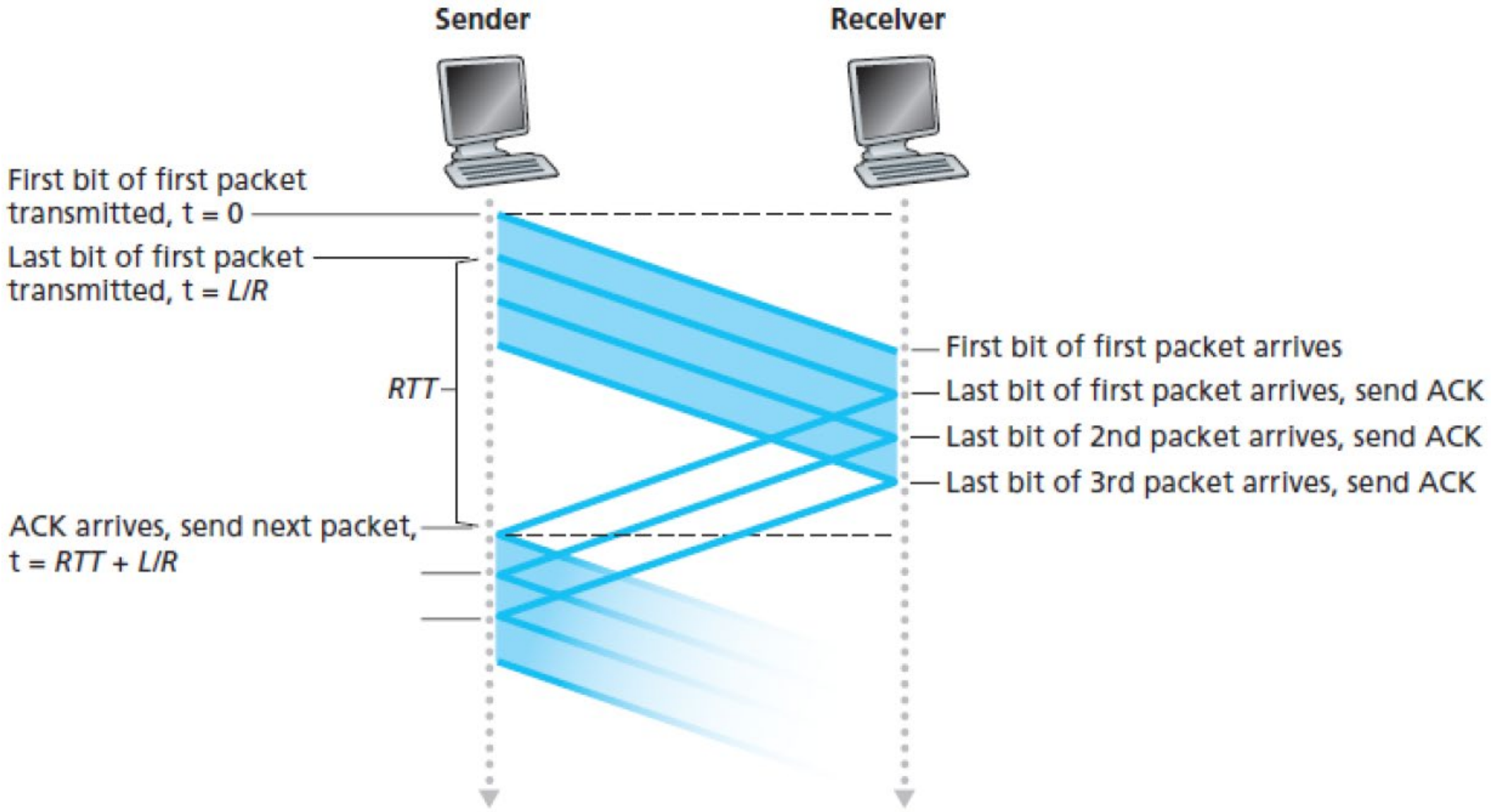
(a) a stop-and-wait protocol in operation

4.13 Rdt 3.0 Stop and Wait Operation !



a. Stop-and-wait operation

4.14 Enhanced Rdt3.0 with Pipelining



b. Pipelined operation

4.15 Consequences on Pipelining

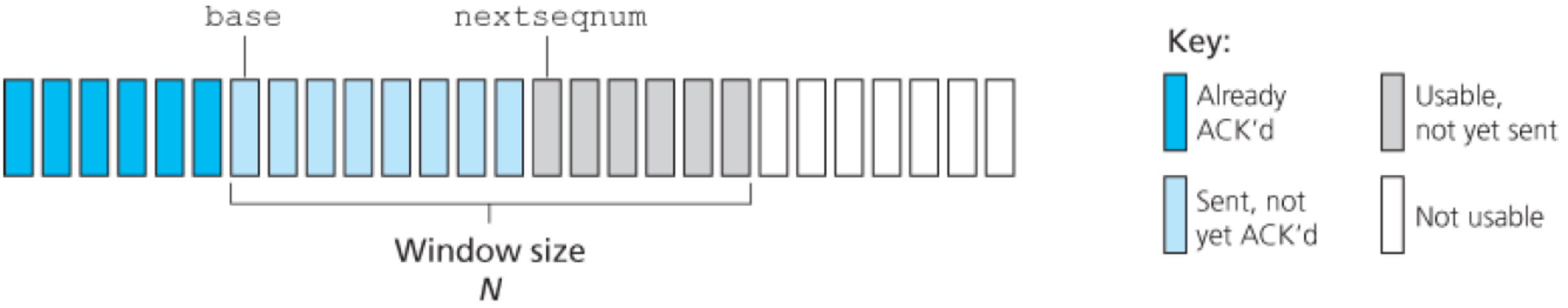
With multiple packets in flight:

- More sequence numbers are needed to distinguish packets.
- The sender buffers unacknowledged packets; the receiver may buffer out-of-order packets.
- The loss-recovery strategy determines how many packets are retransmitted.

- Two basic approaches toward pipelined error recovery can be identified: **Go-Back-N** and **selective repeat**.

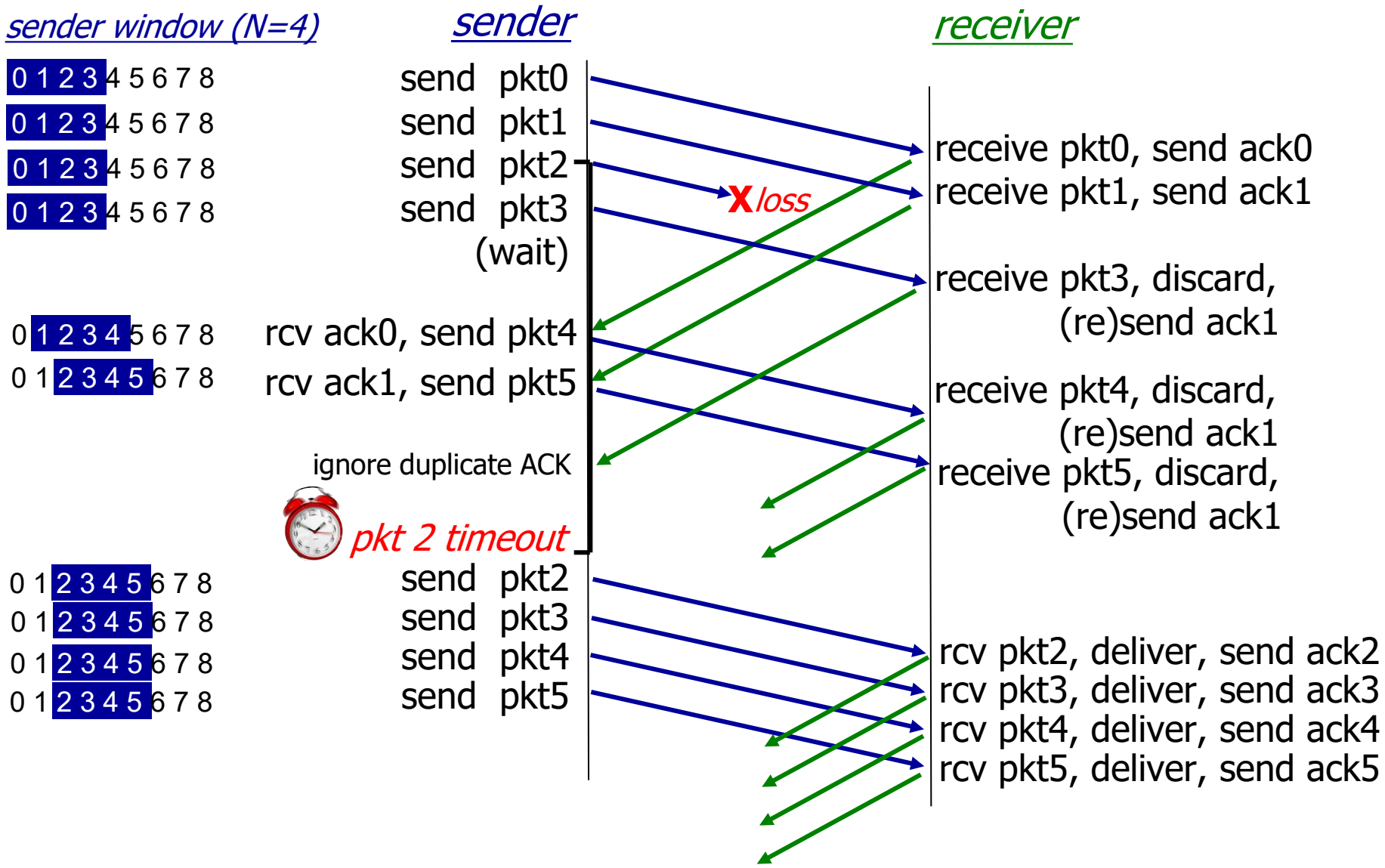
4.16 Go-Back-N (GBN) Protocol

- In a **Go-Back-N (GBN) protocol**, the sender is allowed to transmit multiple packets (when available) without waiting for an acknowledgment, but is constrained to have no more than some maximum allowable number, *N,* of unacknowledged packets in the pipeline.
- Receiver always send ACK for correctly-received packet so far, with highest *in-order* seq #
  - may generate duplicate ACKs
  - can discard (don't buffer) or buffer: an implementation decision



Sender's view of sequence numbers in Go-Back-N

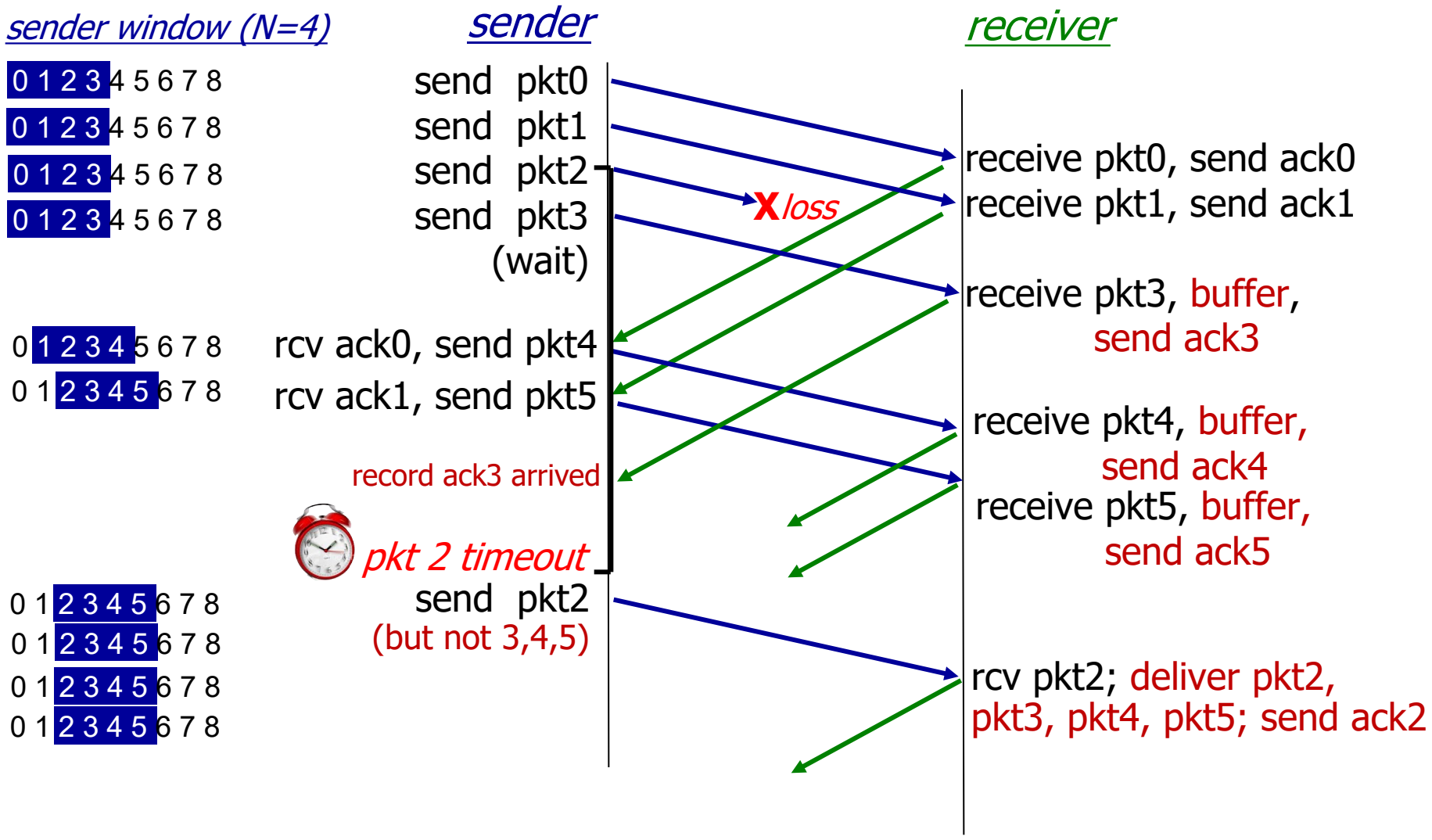
4.17 Go-Back-N in Action



4.18 Selective Repeat Protocol

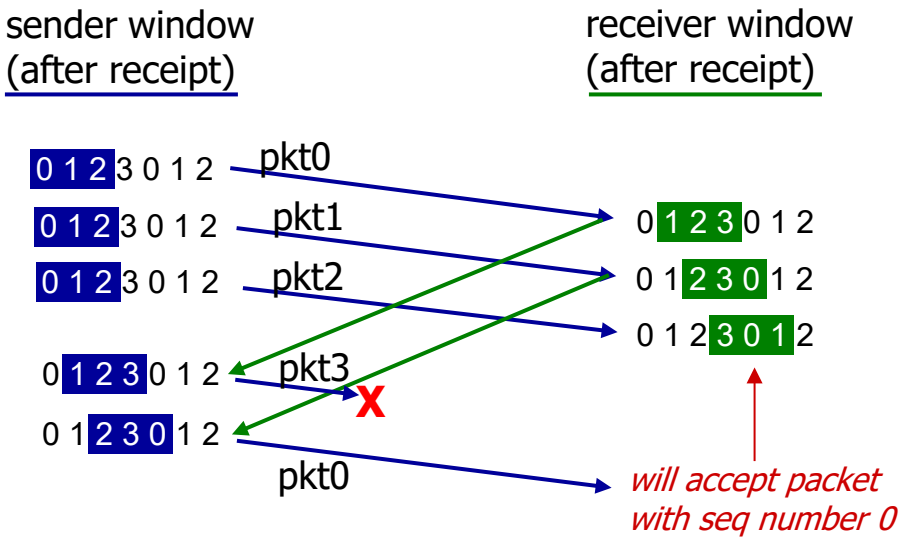
- The GBN protocol allows the sender to potentially “fill the pipeline” with packets, thus avoiding the channel utilization problems we noted with stop-and-wait protocols.
- **Selective Repeat** retransmits only packets that appear lost or corrupted.
- The receiver acknowledges packets individually and buffers out-of-order packets.
- It avoids many unnecessary retransmissions but requires more buffering and receiver logic.

4.19 Selective Repeat in Action

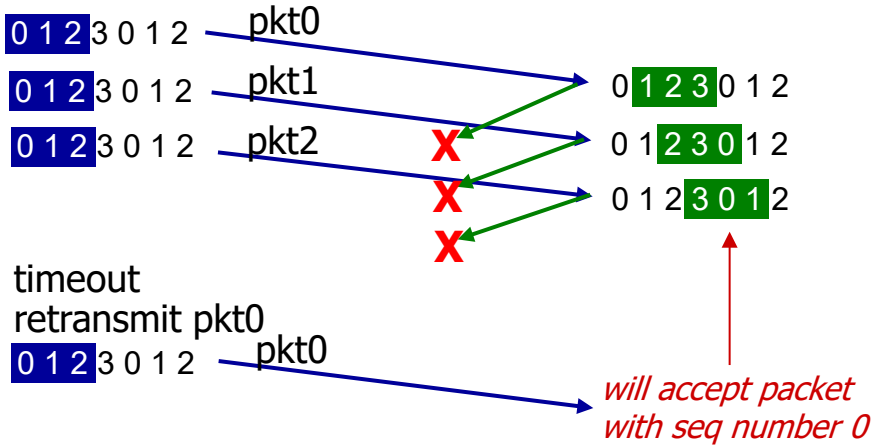


4.20 Selective Repeat: a Dilemma!

- SR receiver dilemma with too-large windows: A new packet or a retransmission?
- seq #s: 0, 1, 2, 3 (base 4 counting); window size=3



(a) no problem



(b) oops!

4.21 Go-Back-N vs Selective Repeat: One Packet Is Lost

Scenario: packet 2 is lost, but packets 3, 4, and 5 reach the receiver.

GO-BACK-N (GBN)

- Receiver typically discards out-of-order packets 3–5.
- After timeout, sender retransmits packet 2 and the later packets in its window.
- Simpler receiver; potentially more redundant traffic.

SELECTIVE REPEAT (SR)

- Receiver buffers packets 3–5 and ACKs them individually.
- Sender retransmits only the missing packet 2.
- Fewer unnecessary retransmissions; more receiver state and buffering.

SR window rule: with m sequence-number bits, the window size must be ≤ 2^(m−1).

5.1 Next Week

- We will:
  - have First Midterm Exam (15:00-16:30 Balat Salon & PC.Lab3)

Your duties:

✓ Review this lecture note via the Learn system.

✓ Read Chapter 3 from Kurose & Ross's textbook.

✓ **Study well for the exam**.

5.2 About First Midterm Exam (20% of the overall grade)

- The exam is expected to consist of approximately 8–10 questions and is planned to last 80–90 minutes.
- The exam will be closed-book and closed-notes (phones). No notebooks, textbooks, or other reference materials will be allowed.
- Calculators are not permitted.
- As this will be a written exam, please remember to bring your own pen.
- Please make sure to arrive on time and be prepared.
- You are responsible for all topics covered from the first week up to and including this week's topics.
- When preparing for the exam, please prioritize the textbook and lecture slides as your primary study materials.

Thank You
For Your Attention!

